

■ COMPLEXITÉ ALGORITHMIQUE

Guide de Révision Complet — M1 ENI Fianarantsoa

Tous les chapitres • Exemples détaillés • Sujets d'examen traités

Année Universitaire 2024-2025 / 2025-2026

■ PLAN DU COURS	■ COMMENT UTILISER CE GUIDE
Chap 1 — Complexité algorithmique Chap 2 — Tableaux et listes chaînées Chap 3 — Piles et files d'attente Chap 4 — Tables de hachage (hash) Chap 5 — Arbres binaires	1. Lis d'abord les encadrés bleus (essentiels) 2. Regarde les exemples avec trace détaillée 3. Entraîne-toi sur les sujets d'examen en bas 4. Mémorise les complexités (tableau récap) 5. Écris les algorithmes clés de mémoire

CHAPITRE 1 — COMPLEXITÉ ALGORITHMIQUE

Rappels, fonctions, notations asymptotiques $O(n)$

1.1 — Pourquoi la complexité ?

La **complexité algorithmique** permet de mesurer la performance d'un algorithme indépendamment du langage, du processeur ou du compilateur. Elle répond à : *combien d'opérations fait cet algorithme pour des données de taille n ?*

■ Complexité temporelle

Nombre d'opérations élémentaires (affectation, comparaison, calcul arithmétique). On compte chaque instruction à coût = 1. On cherche $T(n) = f(n)$.

■ Complexité spatiale

Quantité de mémoire utilisée en fonction de n . Plus rare dans les examens, mais important à mentionner.

1.2 — Notations asymptotiques (O , Ω , Θ)

■ À RETENIR — Les 3 notations fondamentales

$O(f(n))$ = borne supérieure (pire cas). Ex: $T(n) = O(n^2)$ signifie que l'algorithme ne fait **jamais plus** que cn^2 opérations.

$\Omega(f(n))$ = borne inférieure (meilleur cas). Ex: au mieux $T(n) = \Omega(n)$.

$\Theta(f(n))$ = borne exacte (si $O = \Omega$). Utilisé quand c'est précis.

En examen, on utilise principalement $O(n)$ — toujours préciser si c'est pire cas ou meilleur cas.

■ Tableau récapitulatif des complexités

Notation	Nom	Exemple
$O(1)$	Constante	Accès tableau <code>t[i]</code>
$O(\log n)$	Logarithmique	Recherche dichotomique
$O(n)$	Linéaire	Parcours liste/tableau
$O(n \log n)$	Quasi-linéaire	Tri rapide, tri fusion
$O(n^2)$	Quadratique	Tri insertion, 2 boucles imbriquées
$O(n^3)$	Cubique	3 boucles imbriquées
$O(2^n)$	Exponentielle	Fibonacci récursif, sous-ensembles
$O(n!)$	Factorielle	Permutations, problème voyageur

1.3 — Règles de calcul de complexité

Pour calculer $T(n)$, on applique ces règles systématiquement :

Règle	Formule	Exemple
-------	---------	---------

Instruction simple	$T = O(1)$	$a := b + 3 \rightarrow T = O(2) = O(1)$
Séquence	$T = T_1 + T_2$	3 instructions: $T = 3 = O(1)$
Boucle tantque	$T = n \times T_{\text{corps}}$	n itérations: $T = O(n)$
Boucle imbriquée	$T = n \times m \times T_{\text{corps}}$	2 boucles $n \times n$: $T = O(n^2)$
Si...alors	$T = T_{\text{cond}} + T_{\text{instruction}}$	Pire cas: condition vraie
Si...sinon	$T = T_{\text{cond}} + \max(T_1, T_2)$	Pire cas = max des 2 branches
Récursion	$T(n) = T(n-1) + O(1)$	Résoudre l'équation de récurrence

1.4 — Exemples de calcul commentés

Exemple 1 : Boucle simple ($O(n)$)

```
vide afficherN(entier n)
debfonc
entier i;
pour i:=1 haut n faire <-- n itérations
écrire("Algorithme"); <-- 1 opération par itération
finfaire;
finfonc;
```

■ Calcul de complexité

- Déclaration : 0 opération (déclarations non comptées en général)
- Boucle : n itérations $\times$ 1 opération = **n opérations**
- **Total : $T(n) = n \rightarrow T(n) = O(n)$**

Exemple 2 : Double boucle imbriquée ($O(n^2)$)

```
entier somme(vide)
debfonc
entier n, s, a;
n:=100; s:=0; <-- 2 opérations
pour i:=1 haut n faire <-- boucle externe: n fois
a:=n; <-- 1 opération
tantque a>1 faire <-- boucle interne: log(n) fois
a:=a/2; <-- 1 opération
finfaire;
s:=s+1; <-- 1 opération
finfaire;
retourner s;
finfonc;
```

■ Calcul de complexité

- Boucle externe : n itérations
- Boucle interne (tantque $a > 1$, $a := a/2$) : divise a par 2 à chaque fois $\rightarrow$ **$\log_2(n)$ itérations**
- **Total : $T(n) = n \times \log_2(n) \rightarrow T(n) = O(n \log n)$**

Exemple 3 : Fonction récursive — Fibonacci ($O(2^n)$)

C'est exactement le sujet de l'examen 2024-2025 ! Voici la fonction :

```
entier fct(entier n)
debfonc
si n<2 alors retourner n; <-- cas de base : 1 opération
sinon retourner fct(n-1) + fct(n-2); <-- 2 appels récursifs
finfonc;
```

■ SUJET D'EXAMEN — Calcul du nombre d'appels pour n=4 et n=10

Équation de récurrence : $T(n) = T(n-1) + T(n-2) + 1$ (avec $T(0)=T(1)=1$)

Arbre des appels pour n=4 :

fct(4) appelle fct(3) et fct(2)

fct(3) appelle fct(2) et fct(1)

fct(2) appelle fct(1) et fct(0) → 2 appels de base + 1 = 3 appels

fct(1) → 1 appel de base

fct(2) appelle fct(1) et fct(0) → 3 appels

Calcul par la formule : $T(n) = 2 \cdot T(n-1) - T(n-3)$ [Fibonacci]

$T(0)=1, T(1)=1, T(2)=3, T(3)=5, T(4)=9, T(5)=15 \dots$

→ Pour n=4 : $T(4) = 9$ appels

Pour n=10 : $T(6)=25, T(7)=41, T(8)=67, T(9)=109, T(10)=177$

→ Pour n=10 : $T(10) = 177$ appels

Complexité : $T(n) = O(2^n)$ — exponentielle car chaque appel génère 2 sous-appels.

■ SUJET RATTRAPAGE — Factorielle récursive

Fonction : si $n \leq 1$ retourner 1, sinon retourner $n \times \text{fct}(n-1)$

Équation de récurrence : $T(n) = T(n-1) + 1$, avec $T(1) = 1$

Déroulement : $T(n) = T(n-1)+1 = T(n-2)+2 = \dots = T(1) + (n-1) = 1+(n-1) = n$

→ $T(n) = O(n)$ — linéaire

CHAPITRE 2 — TABLEAUX ET LISTES CHAÎNÉES

Structures de données fondamentales

2.1 — Tableaux

■ Définition

Un tableau $t[1..n]$ est une collection d'éléments de même type accessibles par indice. Opérations principales : accès direct $t[i]$, parcours, recherche, tri.

Parcours itératif d'un tableau — $O(n)$

```
vide parcours(d entier t[], d entier n)
debfonc
entier i;
i:=1;
tantque i≤n faire
  traiter(t[i]); <-- action quelconque sur t[i]
  i:=i+1;
finfaire;
finfonc;
// Complexité :  $T(n) = 1 + 3n = O(n)$ 
```

Recherche séquentielle — $O(n)$

```
booléen acces(d entier t[], d entier n, d entier elem)
debfonc
entier i; booléen trouve;
i:=0; trouve:=faux;
tantque (i<n) et (non trouve) faire
  si t[i]=elem alors trouve:=vrai;
  sinon i:=i+1;
finsi;
finfaire;
retourner trouve;
finfonc;
// Complexité :  $O(n)$  — pire cas : elem absent, on parcourt tout
```

Recherche dichotomique (tableau trié) — $O(\log n)$

```
booléen dich(d entier t[], d entier n, d entier elem)
debfonc
entier inf, sup, m; booléen trouve;
trouve:=faux;
inf:=1; sup:=n;
tantque (inf≤sup) et (non trouve) faire
  m:=(inf+sup) div 2; <-- milieu
  si t[m]=elem alors trouve:=vrai;
  sinonsi t[m]<elem alors inf:=m+1; <-- chercher à droite
  sinon sup:=m-1; <-- chercher à gauche
finsi; finsi;
finfaire;
retourner trouve;
finfonc;
```

```
// Complexité :  $O(\log n)$  — divise par 2 à chaque itération
```

■ Pourquoi $O(\log n)$?

À chaque itération, on divise le tableau par 2. Après q itérations, taille = $n/2^q$.

Quand $n/2^q = 1 \rightarrow 2^q = n \rightarrow q = \log_2(n)$. Donc $T(n) = O(\log n)$.

Tris de tableau

Tri par insertion — $O(n^2)$

```
vide triinsertion(dr entier t[], d entier n)
debfonc
entier i, j, cle;
pour i:=2 haut n faire <-- i de 2 à n
cle:=t[i]; <-- élément à insérer
j:=i-1;
tantque j>0 et t[j]>cle faire <-- décaler vers droite
t[j+1]:=t[j];
j:=j-1;
finfaire;
t[j+1]:=cle; <-- insérer au bon endroit
finfaire;
finfonc;
// Complexité pire cas :  $O(n^2)$  — tableau trié à l'envers
```

Tri rapide (quicksort) — $O(n \log n)$ en moyenne

```
vide trisegmentation(dr entier t[], d entier inf, d entier sup)
debfonc
entier place;
si inf<sup alors
segmentation(t, inf, sup, place); <-- choisir pivot
trisegmentation(t, inf, place-1); <-- trier gauche
trisegmentation(t, place+1, sup); <-- trier droite
finsi;
finfonc;

vide segmentation(dr entier t[], d entier inf, d entier sup, r entier place)
debfonc
entier i, j, pivot;
pivot:=t[inf]; i:=inf+1; j:=sup;
tantque i≤j faire
si t[i]≤pivot alors i:=i+1;
sinon permut(t[i], t[j]); j:=j-1;
finsi;
finfaire;
permut(t[inf], t[j]);
place:=j;
finfonc;
// Complexité :  $O(n \log n)$  en moyenne,  $O(n^2)$  pire cas
```

2.2 — Listes linéaires chaînées

■ Structure d'une liste chaînée

Chaque **cellule** contient : **info** (la donnée) + **suivant** (adresse de la cellule suivante).

La liste est accédée via un pointeur **liste** sur la 1ère cellule. Si liste=nil → liste vide.

```
// DÉCLARATION TYPE (à mémoriser !)
structure cellule
chaîne nom; <-- ou tout autre type
structure cellule* suivant; <-- pointeur sur la suivante
fin;
type structure cellule* pointeur;
pointeur liste, p; <-- variables de type pointeur
```

Création d'une liste (à l'envers — depuis la fin)

```
pointeur creerliste(d entier n)
debfonc
pointeur l, p; entier i; chaîne val;
l:=nil; <-- liste vide au départ
pour i:=1 haut n faire
nouveau(p); <-- allouer une cellule
lire(val); <-- saisir la valeur
p->nom:=val;
p->suivant:=l; <-- chaîner avec la liste existante
l:=p; <-- l pointe sur la nouvelle tête
finfaire;
retourner l;
finfonc;
// Résultat : liste créée à l'ENVERS (dernier saisi = tête)
```

Parcours d'une liste

```
// Parcours à l'endroit (1er → dernier)
vide parcours1(d pointeur l)
debfonc
si l≠nil alors
traiter(l->info); <-- traiter AVANT l'appel récursif
parcours1(l->suivant);
finsi;
finfonc;

// Parcours à l'envers (dernier → 1er) [pile implicite !]
vide parcours2(d pointeur l)
debfonc
si l≠nil alors
parcours2(l->suivant); <-- appel récursif AVANT traitement
traiter(l->info);
finsi;
finfonc;
// Complexité des deux :  $T(n) = O(n)$ 
```

Insertion en tête de liste — $O(1)$

```
vide insertete(dr pointeur l, d t elem)
debfonc
pointeur p;
nouveau(p); <-- créer cellule
p->info:=elem;
p->suivant:=l; <-- chaîner avec la liste (liaison a)
l:=p; <-- nouvelle tête (liaison b)
finfonc;
// Complexité :  $T(n) = O(1)$  — constant, ne dépend pas de n
```

Insertion en fin de liste — $O(n)$

```
vide inserfin(dr pointeur l, d t elem)
```

```

debfonc
pointeur der, p;
si l=nil alors insertete(l, elem); <-- cas liste vide
sinon
der:=dernier(l); <-- trouver dernière cellule O(n)
nouveau(p);
p->info:=elem;
p->suivant:=nil;
der->suivant:=p;
fini;
finfonc;
// Complexité : O(n) – à cause de la recherche du dernier

```

Suppression de la tête — O(1)

```

vide supptete(dr pointeur l)
debfonc
pointeur p;
p:=l;
l:=l->suivant; <-- avancer la tête
laisser(p); <-- libérer mémoire
finfonc;
// Complexité : T(n) = O(1)

```

■ SUJET D'EXAMEN — Ex 2.1 : Vecteur + Pile pour afficher à l'envers

Énoncé : Écrire une fonction qui crée un vecteur de n noms, puis une pile (liste chaînée à l'envers) pour afficher la liste à l'envers.

Solution : Une pile est une liste chaînée où insertion/suppression se font en tête (LIFO).

```

// Étape 1 : créer un vecteur de n noms saisis
vide creervec(r chaîne v[], d entier n)
debfonc
entier i;
pour i:=1 haut n faire
lire(v[i]);
finfaire;
finfonc;

// Étape 2 : créer une pile (liste à l'envers) depuis le vecteur
// En insérant chaque élément en tête → la pile contient dans l'ordre inverse
pointeur creerpile(d chaîne v[], d entier n)
debfonc
pointeur p, pile;
pile:=nil;
entier i;
pour i:=1 haut n faire
nouveau(p);
p->nom:=v[i];
p->suivant:=pile; <-- insertion en tête = pile LIFO
pile:=p;
finfaire;
retourner pile;
finfonc;

// Afficher la pile (= affiche dans l'ordre inverse du vecteur)
vide afficherpile(d pointeur pile)
debfonc
pointeur p;

```

```
p:=pile;
tantque p≠nil faire
  écrire(p->nom);
p:=p->suiwant;
finfaire;
finfonc;
```

■ SUJET D'EXAMEN — Ex 2.2 : Table de hachage — compter les noms

Structure : pointeur hash[10] — tableau de 10 listes chaînées (classes 0..9)

Fonction de hachage : adresse = longueur(nom) mod 10

But : compter le nombre de noms dans la classe d'un nom donné.

```
// Compter les noms dans la classe d'un nom passé en argument
entier compter(d chaine nom)
debfonc
  entier adresse, compteur;
  pointeur p;
  adresse:=longueur(nom) mod 10; <-- calcul adresse de hachage
  p:=hash[adresse]; <-- aller à la liste de cette classe
  compteur:=0;
  tantque p≠nil faire
    compteur:=compteur+1;
    p:=p->suiwant;
  finfaire;
  retourner compteur;
finfonc;
// Complexité : O(k) où k = taille de la classe
```

CHAPITRE 3 — PILES ET FILES D'ATTENTE

Structures de données particulières (LIFO / FIFO)

3.1 — Piles (Stack)

■ PILE (Stack) — LIFO	■ FILE (Queue) — FIFO
Last In, First Out Dernier entré = premier sorti. Exemple : pile d'assiettes Op : empiler(val), dépiler(val)	First In, First Out Premier entré = premier sorti. Exemple : file d'attente guichet Op : ajoutelem(val), suppelem(val)

Primitives d'une pile (représentation contiguë)

```
// Variables globales : pile[], sommet, dimpile
vide initpilevide() // Initialiser la pile
defonc sommet:=0; finonc;

booléen pilevide() // Tester si vide
defonc retourner sommet=0; finonc;

booléen pilepleine() // Tester si pleine
defonc retourner sommet=dimpile; finonc;

vide empiler(d t val) // Ajouter au sommet
defonc
si pilepleine() alors écrire("Erreur: pile pleine"); arrêt; finsi;
sommet:=sommet+1;
pile[sommet]:=val;
finonc;

vide dépiler(r t val) // Retirer du sommet
defonc
si pilevide() alors écrire("Erreur: pile vide"); arrêt; finsi;
val:=pile[sommet];
sommet:=sommet-1;
finonc;
```

Pile avec liste chaînée (représentation chaînée)

```
// Variable globale : pile (pointeur)
vide empiler(d t elem) // = insertion en tête
defonc insertete(pile, elem); finonc;

vide dépiler(r t elem) // = suppression en tête
defonc
si pilevide() alors écrire("Erreur: pile vide"); arrêt; finsi;
elem:=pile->info;
supptete(pile);
finonc;

booléen pilevide() // = liste vide ?
defonc retourner (pile=nil); finonc;
```

Application des piles : Expressions postfixées

■ Les 3 formes d'expressions

Préfixe : opérateur AVANT les opérandes. Ex: + A B = (A+B)

Infixe : opérateur ENTRE les opérandes. Ex: A + B (écriture habituelle)

Postfixe : opérateur APRÈS les opérandes. Ex: A B + = (A+B)

Exemples de conversion :

Expression	Préfixe	Postfixe
(A - B) + C	+ - A B C	A B - C +
A * B + C / (D * E)	+ * A B / C * D E	A B * C D E * / +
((A + B) * C)	* + A B C	A B + C *

Évaluation d'une expression postfixée avec une pile

■ Algorithme d'évaluation (règles)

1. Si le symbole est une **variable** → empiler sa valeur
2. Si le symbole est un **opérateur binaire** → dépiler 2 valeurs, calculer, empiler le résultat
3. Si le symbole est '#' → fin, le résultat est au sommet de la pile

■ Exemple de trace : AB*CD+/ avec A=20, B=4, C=9, D=7

Lecture A → empiler 20 | Pile: [20]

Lecture B → empiler 4 | Pile: [20, 4]

Lecture * → dépiler 4 et 20, calculer $20 \times 4 = 80$, empiler 80 | Pile: [80]

Lecture C → empiler 9 | Pile: [80, 9]

Lecture D → empiler 7 | Pile: [80, 9, 7]

Lecture + → dépiler 7 et 9, calculer $9 + 7 = 16$, empiler 16 | Pile: [80, 16]

Lecture / → dépiler 16 et 80, calculer $80 / 16 = 5$, empiler 5 | Pile: [5]

Lecture # → fin. Résultat = 5 ✓

CHAPITRE 4 — TABLES DE HACHAGE

Accès rapide aux données par une fonction de hachage

4.1 — Principe

■ Concept de base

Une **table de hachage** = tableau de listes chaînées. Une **fonction de hachage** $h(\text{clé})$ calcule un indice dans le tableau. Toutes les clés ayant le même indice forment une **classe d'équivalence**.

Complexité idéale : $O(1)$ pour l'accès (si distribution uniforme).

Structure utilisée dans les examens :

```
// 10 classes d'équivalence (adresses 0 à 9)
// Chaque classe = liste chaînée de noms de MÊME LONGUEUR
pointeur hash[10];
// Fonction de hachage : adresse = longueur(nom) mod 10
// Ex: "Jean" (4 lettres) → hash[4 mod 10] = hash[4]
// "Marie" (5 lettres) → hash[5]
// "Rakoto" (6 lettres) → hash[6]
```

■ SUJET RATTRAPAGE — Retourner adresse de la dernière cellule de hash[3]

Énoncé : retourner l'adresse de la cellule contenant le dernier nom de la classe d'adresse 3.

```
pointeur dernierhash3(vide)
debfonc
pointeur p;
p:=hash[3]; <-- partir du début de la classe 3
si p=nil alors retourner nil; finsi; <-- classe vide
tantque p->suivant≠nil faire
p:=p->suivant; <-- avancer jusqu'au dernier
finfaire;
retourner p; <-- retourner l'adresse du dernier
finfonc;
// Complexité :  $O(k)$  où  $k$  = taille de la liste hash[3]
```

CHAPITRE 5 — ARBRES BINAIRES

Structures arborescentes et parcours

5.1 — Définitions et structure

■ Vocabulaire essentiel

Arbre : structure hiérarchique. **Racine** : nœud principal (sommet).

Feuille : nœud sans enfants. **Hauteur** : nombre de niveaux.

Arbre binaire : chaque nœud a AU MAXIMUM 2 fils (gauche et droit).

```
// DÉCLARATION arbre binaire (à mémoriser !)  
structure noeud  
entier info; <-- ou tout autre type  
structure noeud* gauche; <-- sous-arbre gauche  
structure noeud* droite; <-- sous-arbre droit  
fin;  
type structure noeud* pointeur_arbre;  
pointeur_arbre racine;
```

5.2 — Notation et représentation

■ Lire la notation A(B, C) = Arbre

A(B, C) signifie : A est la racine, B est le fils gauche, C est le fils droit.

A(B(D, E), C) signifie : A racine, sous-arbre gauche = B avec fils D et E, sous-arbre droit = C feuille.

■ SUJET D'EXAMEN — Arbre A(B,(D(G)),P(E(M,I),N(F)))

Analysons la notation :

A est la racine

Fils gauche de A = B(D(G)) → B sans fils gauche, fils droit = D(G)

D a un fils gauche = G (feuille)

Fils droit de A = P(E(M,I),N(F))

E a fils gauche M et fils droit I (toutes deux feuilles)

N sans fils gauche, fils droit = F (feuille)

Représentation visuelle :

A

/ \

B P

\ / \

D E N
/ / \ \
G M I F

5.3 — Les 3 parcours d'arbres binaires

■ Les 3 ordres de parcours (à mémoriser !)

Préfixé (Préordre) : Racine → Gauche → Droite (RGD)

Infixé (Inordre) : Gauche → Racine → Droite (GRD) — donne l'ordre trié pour un ABR

Postfixé (Postordre) : Gauche → Droite → Racine (GDR)

■ SUJET D'EXAMEN — Parcours de A(B,(D(G)),P(E(M,I),N(,F)))

Parcours PRÉFIXÉ (RGD) :

A → B → D → G → P → E → M → I → N → F

(On visite chaque racine AVANT ses fils)

Parcours INFIXÉ (GRD) :

B → G → D → A → M → E → I → P → N → F

(On visite le fils gauche, PUIS la racine, PUIS le fils droit)

Parcours POSTFIXÉ (GDR) :

G → D → B → M → I → E → F → N → P → A

(On visite les fils AVANT la racine)

Algorithme récursif de parcours infixé :

```
// Parcours infixé récursif (SUJET D'EXAMEN !)  
vide infixe(d pointeur_arbre a)  
defonc  
si a≠nil alors  
  infixe(a->gauche); <-- 1. visiter sous-arbre gauche  
  écrire(a->info); <-- 2. traiter la racine  
  infixe(a->droite); <-- 3. visiter sous-arbre droit  
finsi;  
finfonc;  
  
// Parcours préfixé récursif  
vide prefixe(d pointeur_arbre a)  
defonc  
si a≠nil alors  
  écrire(a->info); <-- 1. traiter la racine EN PREMIER  
  prefixe(a->gauche);  
  prefixe(a->droite);  
finsi;  
finfonc;  
  
// Parcours postfixé récursif  
vide postfixe(d pointeur_arbre a)
```

```
debfonc
si a≠nil alors
postfixe(a->gauche);
postfixe(a->droite);
écrire(a->info); <-- 1. traiter la racine EN DERNIER
finsi;
finfonc;
// Complexité des 3 parcours :  $O(n)$  – visite chaque nœud 1 fois
```

■ SUJET RATRAPAGE — Déclarer un arbre binaire étudiant

Énoncé : Déclarer le type arbre binaire avec matricule, nom, moyenne. Définir le type pointeur. Écrire une variable pour accéder à la moyenne.

```
// Déclaration de la structure
structure etudiant
entier matricule;
chaîne nom;
réel moyenne;
structure etudiant* gauche;
structure etudiant* droite;
fin;
// Type pointeur correspondant
type structure etudiant* pt_etudiant;
// Variable pour accéder à la moyenne
pt_etudiant e;
// Accès à la moyenne : e->moyenne
// Si e pointe sur un nœud, e->moyenne donne la moyenne de cet étudiant.
```

■ RÉCAPITULATIF FINAL — À MÉMORISER

Les formules et algorithmes clés pour l'examen

Complexités à connaître par cœur

Notation	Nom	Exemple
$O(1)$	Constante	Accès tableau <code>t[i]</code>
$O(\log n)$	Logarithmique	Recherche dichotomique
$O(n)$	Linéaire	Parcours liste/tableau
$O(n \log n)$	Quasi-linéaire	Tri rapide, tri fusion
$O(n^2)$	Quadratique	Tri insertion, 2 boucles imbriquées
$O(n^3)$	Cubique	3 boucles imbriquées
$O(2^n)$	Exponentielle	Fibonacci récursif, sous-ensembles
$O(n!)$	Factorielle	Permutations, problème voyageur

Algorithmes clés et leurs complexités

Algorithme	Complexité	Remarque
Parcours tableau/liste	$O(n)$	Une seule boucle
Recherche séquentielle	$O(n)$	Pire cas: élément absent
Recherche dichotomique	$O(\log n)$	Tableau TRIÉ obligatoire
Tri insertion	$O(n^2)$	Tableau trié à l'envers = pire cas
Tri rapide (quicksort)	$O(n \log n)$	Moyenne; $O(n^2)$ pire cas
Tri fusion	$O(n \log n)$	Toujours $O(n \log n)$
Insertion tête liste	$O(1)$	Constant, pas de parcours
Insertion fin liste	$O(n)$	Chercher le dernier
Suppression tête liste	$O(1)$	Constant
Parcours arbre	$O(n)$	Visite chaque nœud 1 fois
Fibonacci récursif	$O(2^n)$	Exponentiel !
Factorielle récursive	$O(n)$	Linéaire
Hachage (accès)	$O(1)$	Si distribution uniforme

Déclarations types à mémoriser

```
// 1. LISTE CHAÎNÉE
structure cellule
chaîne nom; structure cellule* suivant;
```

```

fin;
type structure cellule* pointeur;
pointeur hash[10]; // Table de hachage
// 2. ARBRE BINAIRE
structure noeud
entier info;
structure noeud* gauche; structure noeud* droite;
fin;
type structure noeud* pointeur_arbre;
// 3. OPÉRATIONS MÉMOIRE
nouveau(p) // Allouer une cellule → p pointe dessus
laisser(p) // Libérer la cellule pointée par p
p->champ // Accéder au champ d'une cellule via pointeur

```

■ STRATÉGIE POUR L'EXAMEN

1. **Complexité** : Identifier le type (récursif/itératif), compter les boucles imbriquées, résoudre la récurrence.
2. **Listes chaînées** : Toujours penser aux cas nil (liste vide). Insérer en tête = $O(1)$.
3. **Pile** : LIFO = insertions/suppressions en tête. empiler = insertete, dépiler = supptete.
4. **Arbres** : Préfixé = Racine d'abord, Infixé = Racine au milieu, Postfixé = Racine à la fin.
5. **Hash** : $h(\text{nom}) = \text{longueur}(\text{nom}) \bmod 10$. Chaque classe = liste de noms même longueur.